

《数据结构》第三次实验报告

雷天乐 2025110528 25 数据科学 1 班

1 实验 1（把顺序栈和栈的应用合在一个实验里了）

实验任务：顺序栈的基础操作和栈的应用

实验时间：1 hours

知识点：顺序栈是用数组实现的栈，遵循”后进先出”（LIFO）原则。可以入栈 push（先判栈满，再将元素放入）出栈 pop（先判栈空）以及初始化等功能

源代码（基础操作 + 数制转换、括号匹配、按行编辑）

```
#include <stdio.h>
#include <stdlib.h>

#define STACK_INIT_SIZE 100
#define STACKINCREMENT 10
#define OVERFLOW -2
#define OK 1
#define ERROR -1
#define TRUE 1
#define FALSE 0
typedef int SElemType;
typedef int Status;
typedef int ElemType;

typedef struct {
    SElemType* base; //栈底指针
    SElemType* top; //栈顶指针
    int stacksize;
} SqStack;

//当前可使用最大容量，不是栈中元素个数
```

```
Status InitStack(SqStack *S) {
    S->base=(SElemType*)malloc(STACK_INIT_SIZE*sizeof(SElemType));
    if (!S->base) exit (OVERFLOW); //存 储 分 配 失 败
    S->top = S->base;
    S->stacksize = STACK_INIT_SIZE;
    return OK;
}

Status Push(SqStack* S, SElemType e){
    if (S->top - S->base >= S->stacksize) {
        S->base = (SElemType *) realloc (S->base,(S->stacksize +
            STACKINCREMENT) * sizeof (SElemType));

        if (!S->base) exit (OVERFLOW);
        S->top = S->base + S->stacksize;
        S->stacksize += STACKINCREMENT;
    }
    *S->top++ = e; // *S->top=e; S->top++;
    return OK;
}

Status Pop(SqStack* S, SElemType *e){
    if (S->top == S->base) return ERROR;
    *e = --S->top; // --S->top; *e=*S->top;
    return OK;
}

Status GetTop (SqStack S, SElemType *e){
    if (S.top == S.base)
        return ERROR;
    *e = *(S.top-1);
    return OK;
}

Status StackEmpty(SqStack S){
    return S.top == S.base;
}

void ClearStack(SqStack *S){
    S->top = S->base; //逻辑清空，内存保留可复用
```

```
}

void DestroyStack(SqStack *S){
    free(S->base);
    S->base = S->top = NULL;
    S->stacksize = 0;
}

void conversion (int N, int d){
    ElemType e;
    SqStack S;
    InitStack(&S); // 构造空栈
    while (N){
        Push(&S, N % d);
        N = N/d;
    }
    while (!StackEmpty(S)){
        Pop(&S, &e);
        printf ("%d", e);
    }
    printf("\n");
    DestroyStack(&S);
} // 数制转换

Status check() {
// 括号匹配的检验算法
    char ch;
    SElemType e;
    SqStack S;
    InitStack(&S);
    while ((ch = getchar()) != '#') {
        if (ch == '(' || ch == '[' || ch == '{')
            Push(&S, ch);
        if (ch == ')')
            if (StackEmpty(S))
                return FALSE;
            else { Pop(&S, &e); if (e != '(') return FALSE; }
        if (ch == ']')
            if (StackEmpty(S))
                return FALSE;
```

```
        else { Pop(&S, &e); if (e != '[') return FALSE; }
    if (ch == '}')
        if (StackEmpty(S))
            return FALSE;
        else { Pop(&S, &e); if (e != '{') return FALSE; }
    }
    if (StackEmpty(S))
        return TRUE;
    else return FALSE;
    DestroyStack(&S);
}

void LineEdit() { //利用字符栈S, 从终端接收一行并传送
    至调用过程的数据区
    int ch; //必须是int才能正确判断EOF
    SElemType e;
    SqStack S;
    InitStack(&S); //构造空栈S
    printf("请输入一行( #:退格; @:清行):\n");
    ch = getchar();
    while (ch != EOF) { //EOF全文结束符WIN:Ctrl+Z;MAC:Ctrl+D
        while (ch != EOF && ch != '\n') {
            if (ch == '#') {
                if (!StackEmpty(S)) //仅当栈非空时退栈
                    Pop(&S, &e);
            } else if (ch == '@') {
                ClearStack(&S); //重置S为空栈
            } else {
                Push(&S, ch); //有效字符进栈
            }
            ch = getchar(); //从终端接收下一个字符
        }
        SElemType *temp = S.base;
        while (temp != S.top) {
            printf("%c", *temp);
            ++temp;
        } //将从栈底到栈顶的栈内字符传送至调用过程
        的数据区
        ClearStack(&S); //重置S为空栈
        printf("\n");
    }
```

```
        if (ch != EOF) {
            printf("请 输 入 一 行( #:退 格; @:清 行):\n");
            ch = getchar();
        }
    }
    DestroyStack(&S);
}

int main(){
    int N, d;
    printf("=== 数制转换 ===\n请输入N和d: ");
    scanf("%d %d", &N, &d);
    conversion(N, d);

    getchar(); // 吃掉上面留下的换行符
    printf("=== 括号匹配 ===\n输入括号串,以#结束: ");
    if (check())
        printf("匹配!\n");
    else
        printf("不匹配!\n");

    printf("=== 行编辑 ===\n");
    LineEdit();
    return 0;
}
```

输出结果

```
PS D:\MinGW64> D:\MinGW64\test.exe
=== 数制转换 ===
请输入N和d: 35 2
100011
=== 括号匹配 ===
输入括号串,以#结束: ({[]})#
匹配!
=== 行编辑 ===
请输入一行(#:退格; @:清行):

请输入一行(#:退格; @:清行):
abc#
ab
请输入一行(#:退格; @:清行):
abc@def
def
请输入一行(#:退格; @:清行):
PS D:\MinGW64> █
```

实验总结

本次实验我实现了顺序栈的基本操作（初始化、入栈、出栈、取栈顶、判空、清空、销毁）并完成了数制转换、括号匹配检验和行编辑三个应用，其“后进先出”的特性在数制转换中实现了余数的逆序输出，在括号匹配中完成了最近开括号的配对，在行编辑中则通过出栈实现退格、清空栈实现整行作废，模拟了输入缓冲区的工作过程。我不仅加深了对栈结构的理解，也巩固了 C 语言指针传参和动态内存管理的知识。

2 实验 2

实验任务：递归算出汉诺塔移动步骤

实验时间：10 mins

知识点：递归的优点是代码简洁优雅，能自然地表达分治、遍历树形结构这类自相似问题的逻辑；缺点是每次函数调用都有栈开销，递归深度过大容易引发栈溢出，且不加记忆化时可能出现大量重复计算，性能往往不如迭代写法。

源代码

```
#include <stdio.h>

int step = 0; // 记录移动步数

void move(char from, int disk, char to)
{
    step++;
    printf("第 %d 步：将 %d 号盘从 %c 移到 %c\n", step, disk, from,
        to);
}

// 将编号 1~n 的 n 个圆盘从 X 搬到 Z 上，Y 用作辅助
void Hanoi(int n, char X, char Y, char Z)
{
    if (n == 1)
        move(X, 1, Z);
    else
    {
        Hanoi(n - 1, X, Z, Y); // 借助 Z，将 X 上 1~n-1 的盘子移到
            Y
        move(X, n, Z);          // 将编号为 n 的盘子从 X 移到 Z
        Hanoi(n - 1, Y, X, Z); // 借助 X，将 Y 上 1~n-1 的盘子移到
            Z
    }
}

int main(void)
{
    int n;
    printf("请输入圆盘个数 n: ");
```

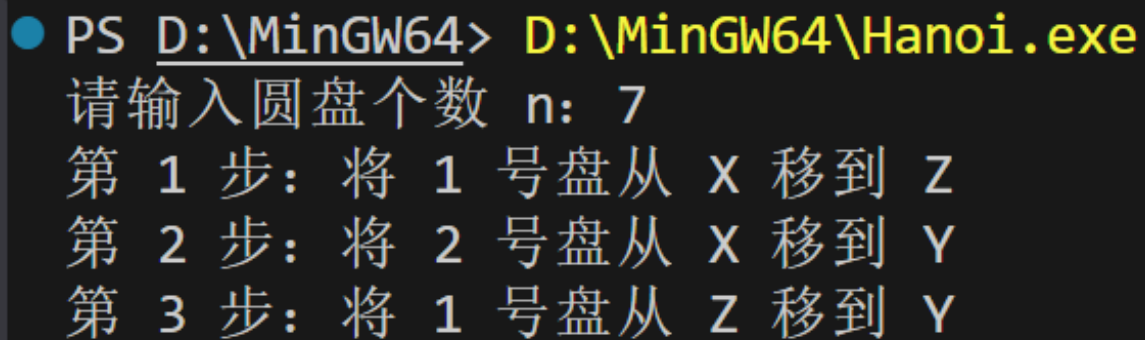
```
scanf("%d", &n);

Hanoi(n, 'X', 'Y', 'Z');

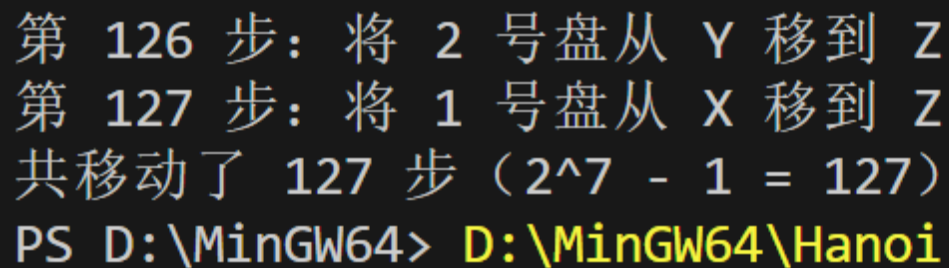
printf("共移动了 %d 步 ( $2^n - 1 = %d$ ) \n", step, n, (1 << n) - 1);

return 0;
}
```

输出结果：（输入 64 的时候一直在打印 2 的 64 次方-1 还是太大了）



```
PS D:\MinGW64> D:\MinGW64\Hanoi.exe
请输入圆盘个数 n: 7
第 1 步：将 1 号盘从 X 移到 Z
第 2 步：将 2 号盘从 X 移到 Y
第 3 步：将 1 号盘从 Z 移到 Y
```



```
第 126 步：将 2 号盘从 Y 移到 Z
第 127 步：将 1 号盘从 X 移到 Z
共移动了 127 步 ( $2^7 - 1 = 127$ )
PS D:\MinGW64> D:\MinGW64\Hanoi.exe
```

实验总结

通过本次汉诺塔递归实验，我理解了递归函数的基本思想以及递归调用的执行过程，加深了对 C 语言函数调用的理解。递归方法代码简洁、思路清晰，但随着盘子数量增加，递归次数会快速增长，效率较低且占用较多栈空间。

3 实验 3

实验任务：循环队列的初始化、入队和出队

实验时间：30 mins

知识点：循环队列采用“先进先出”（FIFO）的原则，初始化时通常将队头 front 和队尾 rear 置为初始位置。入队时将元素放入 rear 所指位置并更新 rear，出队时取出 front 所指元素并更新 front。循环队列通过取模运算实现队尾和队头的循环利用，可以有效避免顺序队列的空间浪费。

源代码

```
#include <stdio.h>
#include <stdlib.h>

#define OK          1
#define ERROR       0
#define OVERFLOW    -2
#define MAXQSIZE    100

typedef int Status;
typedef int ElemType;

typedef struct {
    ElemType *base;
    int front;
    int rear;
} SqQueue;

Status InitQueue(SqQueue *Q)
{
    Q->base = (ElemType *)malloc(MAXQSIZE * sizeof(ElemType));
    if (!Q->base) {
        exit(OVERFLOW);
    }
    Q->front = Q->rear = 0;
    return OK;
}

Status EnQueue(SqQueue *Q, ElemType e)
{

```

```
    if ((Q->rear + 1) % MAXQSIZE == Q->front) {
        return ERROR;
    }

    Q->base[Q->rear] = e;
    Q->rear = (Q->rear + 1) % MAXQSIZE;
    return OK;
}

Status DeQueue(SqQueue *Q, ElemType *e)
{
    if (Q->front == Q->rear) {
        return ERROR;
    }

    *e = Q->base[Q->front];
    Q->front = (Q->front + 1) % MAXQSIZE;
    return OK;
}

int main(){
    SqQueue Q;
    ElemType e;

    InitQueue(&Q);

    for (int x = 10; x <= 50; x += 10) {
        if (EnQueue(&Q, x) == OK) {
            printf("入队 %2d 后: [", x);
            for (int i = Q.front; i != Q.rear; i = (i + 1) % MAXQSIZE) {
                printf("%d%s", Q.base[i], (i + 1) % MAXQSIZE == Q.
                    rear ? "" : ", ");
            }
            printf("] front=%d rear=%d\n", Q.front, Q.rear);
        } else {
            printf("入队 %d 失败: 队列满\n", x);
        }
    }
}
```

```

printf("\n开始出队: \n");
while (DeQueue(&Q, &e) == OK) {
    printf("出队 %2d 后剩: [", e);
    for (int i = Q.front; i != Q.rear; i = (i + 1) % MAXQSIZE) {
        printf("%d%s", Q.base[i], (i + 1) % MAXQSIZE == Q.rear ?
            "" : ", ");
    }
    printf("] front=%d rear=%d\n", Q.front, Q.rear);
}

free(Q.base);
return 0;
}

```

输出结果 (入队 10, 20, 30, 40, 50 后出队 10, 20, 30, 40, 50)

```

● PS C:\Users\TIANLE\Desktop\c> & 'c:\Users\TIANLE\.vscode\ex
.exe' '--stdin=Microsoft-MIEngine-In-ofibjb5f.gfa' '--stdout
2e' '--pid=Microsoft-MIEngine-Pid-qjt5srjc.4pf' '--dbgExe=D
入队 10 后: [10] front=0 rear=1
入队 20 后: [10, 20] front=0 rear=2
入队 30 后: [10, 20, 30] front=0 rear=3
入队 40 后: [10, 20, 30, 40] front=0 rear=4
入队 50 后: [10, 20, 30, 40, 50] front=0 rear=5

开始出队:
出队 10 后剩: [20, 30, 40, 50] front=1 rear=5
出队 20 后剩: [30, 40, 50] front=2 rear=5
出队 30 后剩: [40, 50] front=3 rear=5
出队 40 后剩: [50] front=4 rear=5
出队 50 后剩: [] front=5 rear=5
● PS C:\Users\TIANLE\Desktop\c> 

```

3.1 实验总结

本次实验完成了循环队列的初始化、入队和出队。通过 front、rear 指针与取模运算实现下标回绕,避免了顺序队列的假溢出。入队以 $(rear+1)\%MAXQSIZE==front$ 判满,出队以 $front==rear$ 判空。实验输出 10 20 30 40 50,验证了先进先出特性,也加深了对判空判满和边界处理的理解。